■■ ScamSentry AI

Product Requirements Document (PRD)

Tagline: Check before you trust. Report before others fall victim.

1. Problem Statement

Scams involving phone numbers, UPI IDs and suspicious links can spread quickly through communities. People often have no fast way to check whether an identifier has already been reported. ScamSentry AI is designed as a community-powered scam intelligence platform for India where users can anonymously report suspicious activity, check identifiers before trusting them, view recent area-wise activity, and receive alerts when unusual scam patterns emerge.

2. Product Goal

Build a working hackathon MVP that makes scam reporting quick, identifier lookups fast, community intelligence useful, and abuse-resistant. The system must protect reporter privacy and avoid allowing report-bombing or spam to falsely label legitimate people or businesses as scammers.

3. Core MVP Scope

Feature	Requirement
Check Before You Trust	Look up Phone Numbers, UPI IDs and URLs/Links.
Anonymous Reporting	Allow users to submit suspected scam reports quickly, targeting under 60 seconds.
Fast Lookup	Return a normalized identifier and community risk intelligence quickly.
Community Scam Map	Show the last 7 days of reports grouped by area.
Unusual Activity Alerts	Alert only when scam activity shows an unusual spike or new pattern.
Anti-Abuse Protection	Defend against spam, duplicate reports, fake reports and report-bombing.

4. Official Backend API Surface

Method	Endpoint	Purpose
GET	/health	Service health check.
POST	/reports	Submit an anonymous scam report.
GET	/lookup	Check a phone number, UPI ID or URL.
GET	/map	Return recent area-wise scam activity.
GET	/alerts	Return active unusual-activity alerts.

POST	/admin/reset	Administrative reset endpoint for the hackathon environment.
GET	/license	Read current licence status.
POST	/license	Upload/update licence without restarting the server.

5. Lookup Requirements

The system should accept valid phone numbers, UPI IDs and URLs, normalize the input, and return a verdict based on available community intelligence. Valid but unreported identifiers should return a successful NO_REPORTS-style result rather than being treated as a missing resource. Invalid values should be rejected with appropriate validation errors.

Target verdict categories: KNOWN_SCAM, SUSPICIOUS, NO_REPORTS, VERIFIED_BUSINESS.

6. Trust and Anti-Abuse Requirements

• Repeated reports from the same reporter should not artificially inflate trust or report counts.

• Many accounts originating from the same network/IP context should not automatically be treated as fully independent reporters.

• Duplicate or near-duplicate reports should be detected and handled appropriately.

• Report-bombing attempts must not easily cause a legitimate identifier or business to become KNOWN_SCAM.

• Invalid, junk and clearly malformed input must be rejected.

• Only accepted/trusted reports should contribute to risk calculations, maps and anomaly alerts.

7. Privacy Requirements

Reporter privacy is a core product requirement. The application must not expose reporter identity, reporter contact details, reporter IP addresses or exact reporter locations. Area-level information may be used for aggregation and mapping. Risk messaging should communicate community reports and confidence rather than making unsupported defamatory claims.

Phone-number display should support masking when confidence or trusted independent reporting is below the required threshold.

8. Map Requirements

The map focuses on the most recent seven days of accepted scam reports. Results are aggregated by area rather than exposing a reporter's exact location. Each area can display report counts, relevant scam categories, risk level and anomaly information.

9. Alert Logic

SPIKE: Trigger when reports in the last 24 hours are at least twice the daily average of the previous seven days and the minimum report threshold is reached.

NEW_TYPE: Trigger when a scam type appears in the last 24 hours that was not observed in the same area during the previous 30 days.

Rejected, duplicate or untrusted reports must not be allowed to trigger alerts.

10. Licence Behaviour

The service includes a licence mechanism suitable for hackathon evaluation. Licence updates should take effect without a server restart. When a licence is expired, protected application functionality should return a clear LICENSE_EXPIRED response while health/licence management endpoints remain available.
Existing data must not be deleted because of licence expiry.

11. Team Responsibilities

Team Member	Primary Responsibility
Mohd Avaish	Backend, database, APIs, trust/anti-abuse logic, anomaly detection, integration and deployme
Sartaj	Frontend, UI/UX, React components, lookup/report flows, map/dashboard and visual polish.

12. Recommended Build Order

1. Project foundation and official API contract
2. Minimal FastAPI server and /health
3. Database schema and persistence
4. Anonymous reporting plus trust/anti-abuse rules
5. Lookup and risk verdict logic
6. Seven-day map aggregation
7. Spike and new-type alerts
8. Licence functionality
9. Frontend integration
10. End-to-end testing, deployment, demo and pitch

Document prepared for the CyberForge hackathon project workspace. This PRD is intended as the team's implementation reference.